Variational Auto encoder for Shoes Image Reconstruction

José Luis Almendarez Gonzalez

jose.almendarez@iteso.mx

José Ivan Sandoval Ruiz Velasco

ivan.sandoval@iteso.mx

Maximiliano Aguayo Villanueva

maximiliano.aguayo@iteso.mx

ITESO university

February 26, 2026

Abstract

This paper presents the design, implementation, and evaluation of a Convolutional Variational Autoencoder (CVAE) applied to shoe image reconstruction and latent-space generation. The model is trained on a dataset of shoe images collected via web scraping from StockX, resized to 64×64 pixels, and normalized to $[0, 1]$. We investigate the effect of the β hyperparameter on the trade-off between reconstruction fidelity and generative quality of the latent space. Training is conducted on Apple M2 unified memory using TensorFlow, with a Cosine Decay learning rate schedule. Two dataset configurations are compared: a small set of 177 images and an extended set of 1,220 images. Results show that dataset size does not significantly affect the model's parametrization in terms of reconstruction loss, though it can improve the consistency of latent space representations when exploring latent trajectories. Additionally, $\beta = 1.0$ (standard VAE) provides a reasonable balance between reconstruction loss and KL regularization. Future work proposes the incorporation of perceptual loss functions and adaptive learning rate schedulers to further improve generative quality.

Keywords: Variational Autoencoder, image generation, latent space, shoe dataset, deep learning, TensorFlow

Table of Contents

1 Introduction **3**

2 Mathematical Basis and Network Architecture Design **3**

2.1 ELBO Loss 3

2.2 Reparameterization Trick 4

2.3 Network Architecture 4

3 Experimental Methodology **5**

3.1 Dataset Description and Analysis 5

3.1.1 Shoes 5

3.1.2 Analysis 5

3.1.3 Preprocessing 5

3.2 Implementation Setup 6

3.2.1 Code 6

3.2.2 Memory consideration 6

3.3 Training Procedure 7

3.4 Evaluation Metrics 7

3.4.1 Reconstruction Loss 7

3.4.2 KL Divergence Loss 7

4 Results **7**

5 Conclusions **10**

5.1 Future Work 10

References **10**

1 Introduction

This work goes beyond conducting a Variational Auto-encoder (VAE) experiment and reporting its results. Instead, it proposes a structured and reproducible academic framework for the design, implementation, and evaluation of neural-network models. The framework spans the full development pipeline: architectural design decisions, implementation under Tensor-flow best practices, and systematic analysis of computational resource usage during training.

In an era where machine learning and engineering knowledge and implementations are widely and easy accessible. The challenge is no longer just build models, but to really know well the process behind it. Methodological transparency, and interpretability become essential components of responsible research. Without a structured approach, experiments risk becoming isolated technical exercises rather than cumulative scientific contributions.

This paper therefore emphasizes the importance of consolidating knowledge through a consistent design methodology. By explicitly documenting architectural reasoning, implementation versioning, and memory behavior during training, we aim to provide a replicable and educational blueprint for neural network development. Such a framework not only supports experimental validation but also encourages deeper internalization of computational and theoretical principles. Corregir, ser mas humilde y no asumir

2 Mathematical Basis and Network Architecture Design

A Variational Autoencoder (VAE) is a generative model that learns a probabilistic mapping between an input space $\mathcal{X}$ and a continuous latent space $\mathcal{Z}$. Unlike a standard autoencoder, the encoder does not map inputs to a single point in latent space, but rather to a distribution $q_\phi(z \mid x)$, typically parameterized as a multivariate Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}(z;\, \mu_\phi(x),\, \sigma_\phi^2(x)\mathbf{I}) \tag{1}$$

The decoder then learns the conditional distribution $p_\theta(x \mid z)$, which reconstructs the input from a sampled latent vector.

2.1 ELBO Loss

Training a VAE maximizes the Evidence Lower Bound (ELBO), which decomposes into two terms:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - D_{\mathrm{KL}}\left(q_\phi(z \mid x) \parallel p(z)\right) \tag{2}$$

The first term is the reconstruction loss, which measures how well the decoder recovers the input. In our implementation this is the mean squared error (MSE) summed over pixels:

$$\mathcal{L}_{\text{recon}} = \mathbb{E}\left[\sum_i (x_i - \hat{x}_i)^2\right] \tag{3}$$

The second term is the KL divergence between the approximate posterior and the prior $p(z) = \mathcal{N}(0, \mathbf{I})$, which regularizes the latent space:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{4}$$

Following the β-VAE formulation [**?**], the total loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} \tag{5}$$

where $\beta \geq 1$ controls the strength of KL regularization. Higher β encourages a more structured latent space at the cost of reconstruction fidelity.

2.2 Reparameterization Trick

To enable backpropagation through the stochastic sampling step, the reparameterization trick is applied:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}) \tag{6}$$

This decouples the randomness from the learnable parameters, making gradient computation tractable.

2.3 Network Architecture

The encoder consists of five convolutional layers with stride 2, progressively increasing the number of filters from 32 to 512, followed by a fully connected layer of 256 units. The output is split into two heads: μ_ϕ and $\log\sigma_\phi^2$, each of dimension $d = 128$.

The decoder mirrors this structure using transposed convolutions, starting from a $2 \times 2 \times 512$ spatial representation and upsampling to $64 \times 64 \times 3$. A sigmoid activation in the final layer constrains outputs to $[0, 1]$.

Table 1: Encoder architecture summary.

Layer	Output Shape	Parameters
Conv2D(32, 3, stride=2)	$32 \times 32 \times 32$	896
Conv2D(64, 3, stride=2)	$16 \times 16 \times 64$	18,496
Conv2D(128, 3, stride=2)	$8 \times 8 \times 128$	73,856
Conv2D(256, 3, stride=2)	$4 \times 4 \times 256$	295,168
Conv2D(512, 3, stride=2)	$2 \times 2 \times 512$	1,180,160
Flatten + Dense(256)	256	524,544
Dense(μ), Dense($\log \sigma^2$)	128 each	32,896

3 Experimental Methodology

3.1 Dataset Description and Analysis

3.1.1 Shoes

The dataset was constructed by collecting shoe images from StockX through automated web scraping. The scraping process was designed to retrieve publicly available product images corresponding to different shoe models. These models include various types of footwear such as boots, sport shoes, sandals, and high heels.

3.1.2 Analysis

During the analysis, we examined the distribution of image sizes and color profiles in the dataset. The width of all images was consistently 800 pixels, while the height of most images was 571 pixels, except for 22 images, which ranged from 533 to 991 pixels, representing only 0.018% of the dataset. Regarding color, we extracted the RGB profiles using the colorgram.py tool, which computes a representative color for each image by averaging the values across all pixels in each channel, capturing the main tonality. This approach also allows us to plot the density distribution of the dataset by color components.

3.1.3 Preprocessing

In the preprocessing phase, the RGB color representation of each image was preserved, providing three channels per pixel. All images were resized to 64×64 pixels to standardize the input resolution and reduce computational cost. Pixel values were normalized to the range $[0, 1]$ by dividing by 255. The dataset was split such that training was performed on either 177 or 1,220 images depending on the experiment.

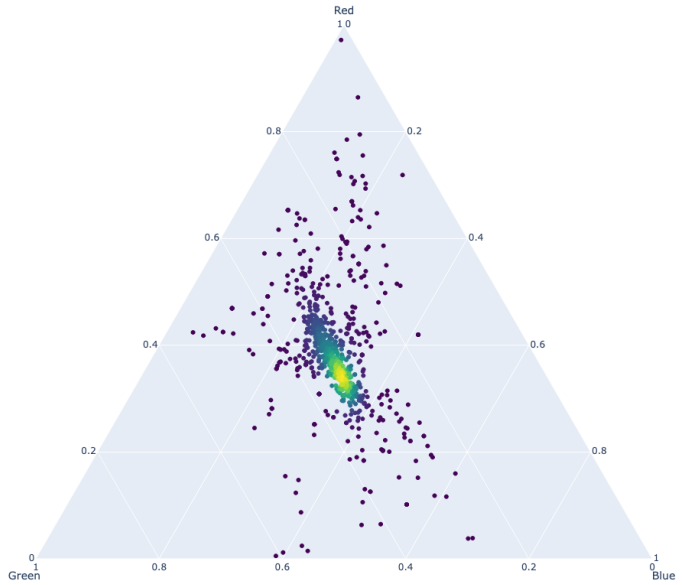

Figure 1: Distribution of the RGB color profiles in the dataset.

3.2 Implementation Setup

3.2.1 Code

The model was implemented in TensorFlow 2.16.2 using the Keras subclassing API. The `ConvVAE` class overrides `train_step` and `test_step` to compute the full ELBO loss internally, without relying on Keras's built-in `compile(loss=...)` mechanism. This ensures that both $\mathcal{L}_{\text{recon}}$ and $\mathcal{L}_{\text{KL}}$ receive gradients correctly in every training step. The sampling layer implements the reparameterization trick as a custom `tf.keras.layers.Layer`.

3.2.2 Memory consideration

All experiments were conducted on an Apple M2 processor with unified memory architecture. In this setting, CPU and GPU share the same physical memory pool which means the same datapath and clock, which means that VRAM and RAM are not independently measurable through standard TensorFlow APIs. The `tf.config.experimental.get_memory_info('GPU:0')` call returns zero under this configuration, as TensorFlow does not expose Apple's Metal backend memory statistics through this interface. Consequently, memory usage was monitored at the process level using `psutil`, which reported RAM consumption in the range of 400–900 MB during training. Full computational profiling with CUDA metrics is deferred to future work on NVIDIA hardware.

3.3 Training Procedure

The optimizer used in all experiments is Adam [**?**]. The learning rate follows a Cosine Decay schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\left(\frac{\pi \cdot t}{T}\right)\right) \tag{7}$$

where $\eta_0 = 10^{-3}$ is the initial learning rate, $\eta_{\min} = 10^{-6}$ is the minimum, t is the current step, and T is the total number of training steps. This schedule provides a smooth decay that avoids abrupt learning rate changes and has been found empirically effective for this architecture.

Experiment 1: Small Dataset (177 images, $\beta = 1.0$)

The model was trained for 2000 epochs on 177 images with batch size 32, yielding 6 batches per epoch. The standard VAE objective ($\beta = 1.0$) was used. This experiment serves as a baseline to evaluate model behavior.

Experiment 2: Extended Dataset (1,220 images, $\beta = 1.0$)

The model was trained for 500 epochs on 1,220 images with the same architecture and hyperparameters. This experiment evaluates whether increased dataset size produces measurably better reconstruction and generative quality.

3.4 Evaluation Metrics

3.4.1 Reconstruction Loss

The primary metric is the reconstruction loss $\mathcal{L}_{\text{recon}}$, computed as the mean squared error summed over all pixels and averaged over the batch. A lower value indicates higher fidelity between input and reconstructed images. This metric is monitored per epoch and serves as the main indicator of model convergence.

3.4.2 KL Divergence Loss

The KL loss $\mathcal{L}_{\text{KL}}$ measures the degree of regularization applied to the latent space. An increasing KL loss during training indicates that the encoder is learning to use the latent dimensions meaningfully, rather than collapsing to the prior. However, excessively high KL values relative to reconstruction loss may indicate posterior collapse or insufficient reconstruction capacity.

4 Results

The small dataset achieves a lower absolute reconstruction loss due to overfitting — the model memorizes 177 images effectively. The extended dataset shows higher absolute loss but generalizes better, producing visually coherent reconstructions with recognizable shoe shapes and

Table 2: Final training metrics with $\beta = 1.0$.

Dataset size	Epochs	Total loss	Reconstruction loss	KL loss
177 images	2000	25.21	13.03	12.19
1,220 images	500	84.10	68.87	15.23

colors across unseen batch samples.

Visual inspection of reconstructed images confirms that the extended dataset model captures geometric structure (silhouette, sole, overall shape) more robustly, while the small dataset model produces sharper reconstructions that are closer to memorization than generalization.

Generated images sampled from the prior $\mathcal{N}(0, \mathbf{I})$ show recognizable shoe shapes in both experiments, confirming that the KL regularization successfully constrains the latent space. However, generated images exhibit blurriness and color mixing artifacts, consistent with the known limitation of MSE-based reconstruction in VAEs.

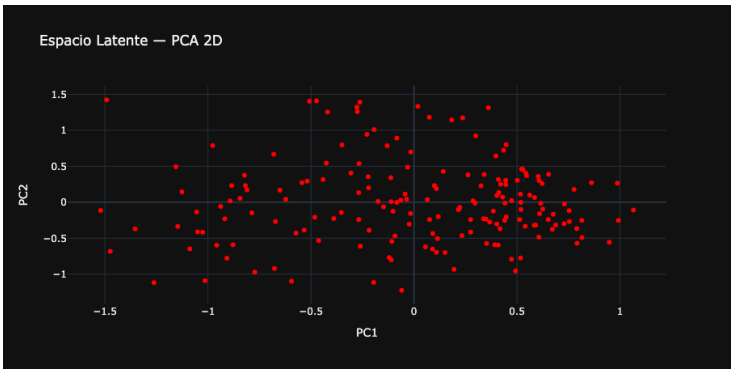

Figure 2: Visualization of the latent space for the 177-image dataset using PCA.

Figure 3: Alternate view of the latent space as interpolation between data points.

Figure 4: Comparison between original shoe images and their reconstructions by the CVAE.

Figure 5: Example of a novel shoe image generated from the latent space.

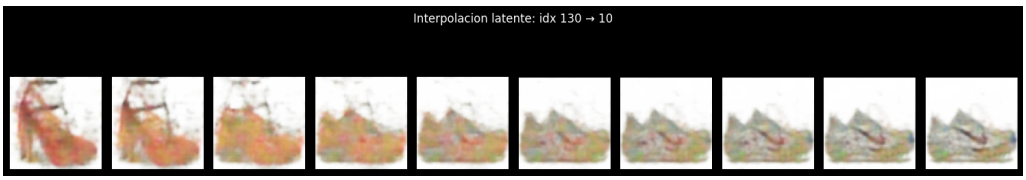

Figure 6: Interpolation between two shoe types in the latent space (high heel to sneaker).

5 Conclusions

This work demonstrates that a Convolutional VAE can learn meaningful latent representations of shoe images for both reconstruction and generation tasks. The standard VAE objective ($\beta = 1.0$) provides a reasonable balance between reconstruction fidelity and latent space regularity. Dataset size has a significant impact on generalization — increasing from 177 to 1,220 images produces visibly more coherent structured latent spaces.

The primary limitation observed is blurriness in generated images, which is an inherent consequence of using pixel-wise MSE as the reconstruction objective. MSE encourages the model to produce the statistical average of plausible outputs, which manifests as blurry, color-mixed images when sampled from the prior.

5.1 Future Work

Several directions are identified for improving generative quality:

Perceptual loss. Replacing or augmenting the MSE reconstruction loss with a perceptual loss computed in the feature space of a pretrained network (e.g., VGG) would penalize structural and textural differences rather than pixel-level discrepancies. This is expected to significantly reduce blurriness in reconstructions and generations.

Adaptive learning rate scheduling. A custom adaptive scheduler that adjusts the learning rate based on loss magnitude and trend — rather than a fixed decay schedule — could improve convergence behavior, particularly under data-scarce conditions where the loss landscape is noisy. Future experiments will evaluate a three-phase adaptive scheduler (acceleration, exploration, fine-tuning) against the Cosine Decay baseline.

CUDA profiling. All experiments in this work were conducted on Apple M2 unified memory. Future work will replicate experiments on NVIDIA hardware to enable full computational profiling, including GPU utilization, VRAM consumption, and energy efficiency metrics measured in FLOPs per watt.

Dataset expansion. Scaling the dataset beyond 1,220 images is expected to yield the largest quality improvement. With 5,000 or more images, the VAE would have sufficient density in the latent space to produce sharp and diverse generations.

Acknowledgments

The authors would like to thank their pets for providing emotional support throughout this work. We also extend our gratitude to Professor Ivan for his clear explanations and foundational guidance, which greatly facilitated the completion of these experiments.

References

[1] TensorFlow Developers. (2024). *TensorFlow* (Version 2.x). https://www.tensorflow.org

[2] Chollet, F., et al. (2024). *Keras*. https://keras.io

[3] Plotly Technologies Inc. (2015). *Collaborative data science*. https://plotly.com

[4] StockX. (2024). *StockX marketplace*. https://stockx.com

[5] TensorFlow. (2024). *tf.keras.optimizers.schedules.CosineDecay*. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/schedules/CosineDecay